A Survey of LLM x Data

Section 1.2 - Techniques of LLM4DATA

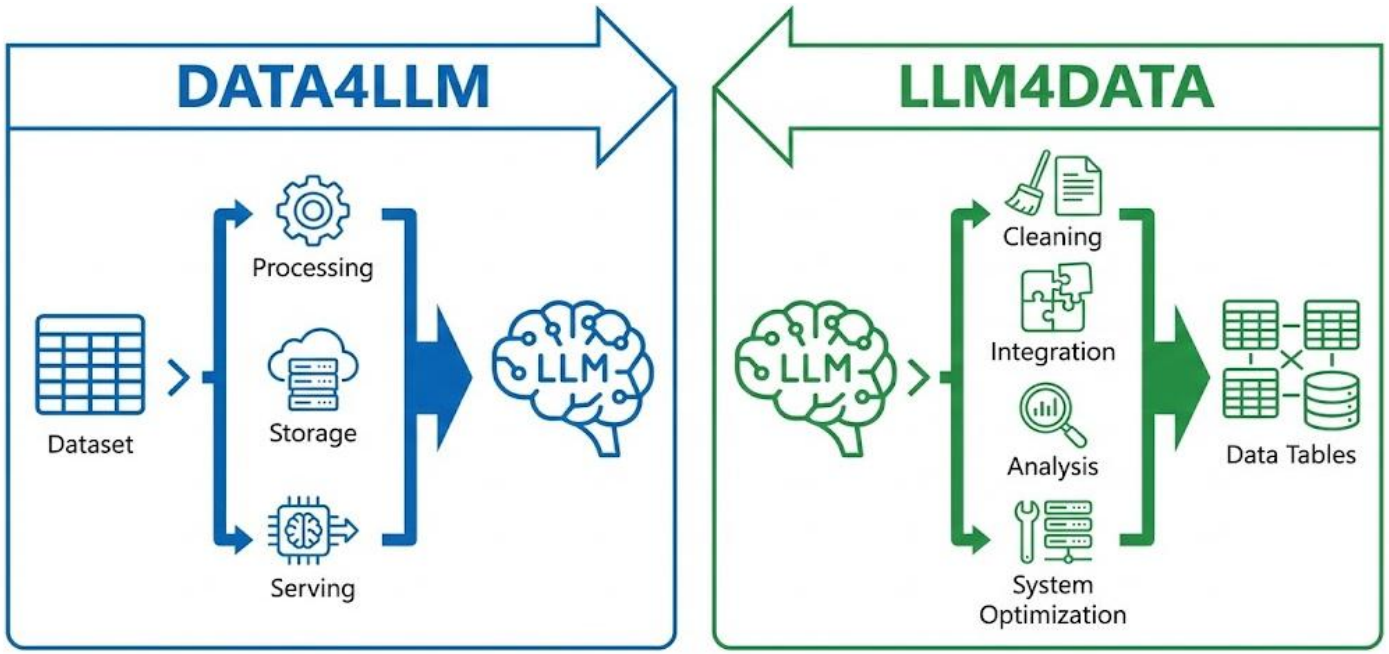
Altan Ulaş Zöhre

CENG 544 Large-Scale Data Management

17.12.2025

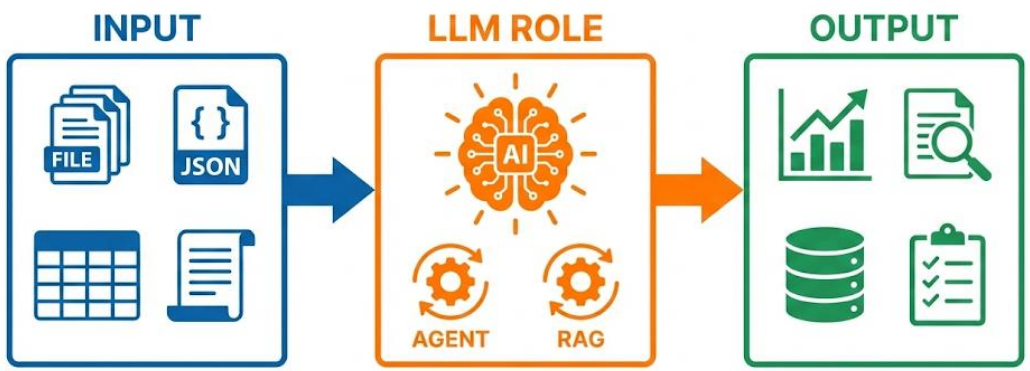
Survey Overview and Scope

- This paper is a survey on the intersection of **LLMs** and **data management / data systems**.
- It organizes the landscape into two directions: **DATA4LLM** and **LLM4DATA**.
- This presentation covers Section 1.2 — **LLM4DATA techniques.**



- **DATA4LLM:** Processing, storing, and serving data to train and run LLMs.
- **LLM4DATA:** Using LLMs for data cleaning, integration, analysis, and system optimization.

How Does It Work?



- **Input:** tables / JSON / documents / code / logs / schemas
- **LLM role:**
  - **Prompting** (natural-language instructions)
  - **RAG** (retrieve external knowledge + answer)
  - **Agent** (plan → execute → verify)
  - *(if needed)* **fine-tuning**
- **Output:** cleaned data, entity matches, analysis answers, optimization recommendations

In this talk, we will ask the same questions for every task:
"What is the input? What does the LLM do? What is the output? (and if needed, which tools does it use?)"

Section 1.2
Technical Taxonomy of LLM4DATA

- **LLMs for Data Manipulation**
  (data preparation: cleaning / integration / discovery)

- **LLMs for Data Analysis**
  (analysis over structured / semi-structured / unstructured data)

- **LLMs for Data System Optimization**
  (system improvement: tuning / query optimization / anomaly diagnosis)

Key idea: *LLMs do not only consume data; they can also serve as a general-purpose component for data management tasks.*

Data Manipulation

- Cleaning
  (Standardize, Correct, Impute)
- Integration
  (Entity Matching, Schema Matching)
- Discovery
  (Profiling, Annotation)

Data Analysis

- Structured
  (Tables/Graphs)
- Semi-Structured
  (JSON/HTML/XML/Spreadsheets)
- Unstructured
  (Documents OCR-dependent vs OCR-free, Code/Repos)

Data System Optimization

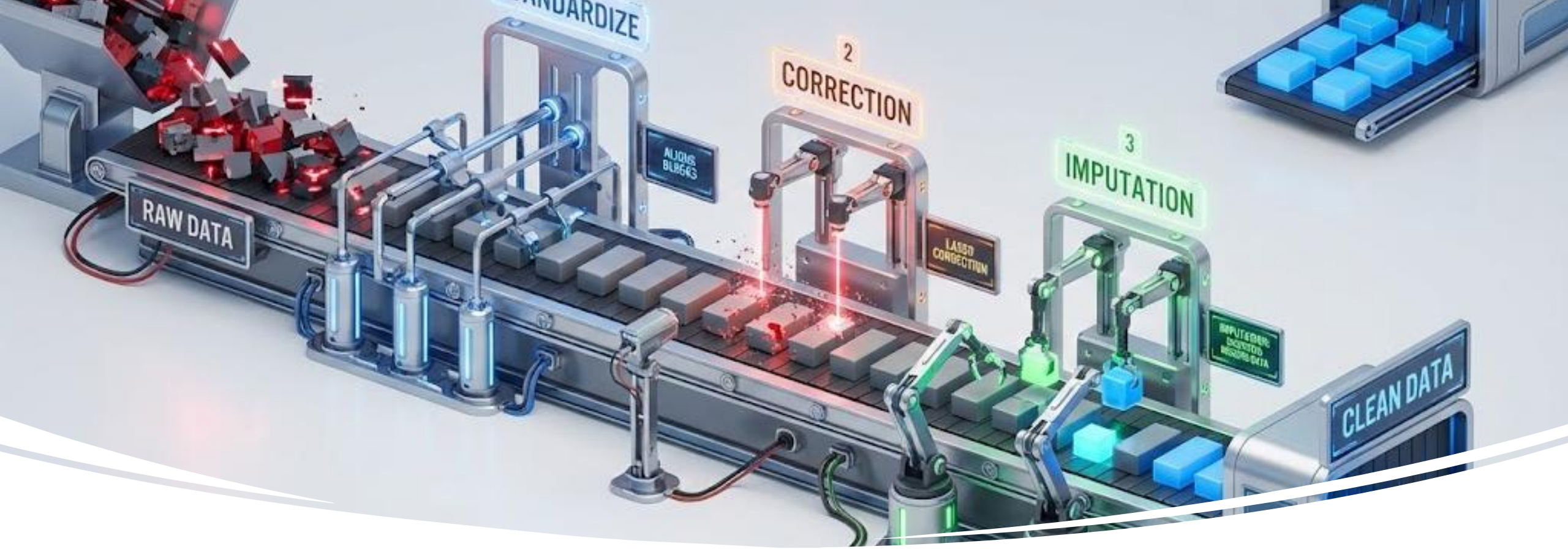
- Configuration Tuning
- Query Optimization
- Anomaly Diagnosis

LLM for Data Manipulation

Goal: prepare data for downstream use

Three sub-tasks:

- **Data Cleaning** *(standardization, error correction, imputation)*
- **Data Integration** *(entity matching, schema matching)*
- **Data Discovery** *(profiling, annotation)*



Data Cleaning

1. **Standardization:** unify formats and representations
2. **Error Processing:** detect and correct incorrect values
3. **Imputation:** fill missing values using context

- Input: noisy / messy data → Output: consistent, more accurate, and more complete data

Data Integration

Goal: unify multiple sources into a consistent view of the same domain

Two core sub-tasks:

1. **Entity Matching:** identify whether records refer to the same real-world entity *(company/person/product)*

2. **Schema Matching:** align fields/columns that represent the same concept *(e.g., "custNo" ↔ "customer_id", "addr" ↔ "address")*

Why it's hard: spelling/abbreviations, missing values, inconsistent formats, different ID logic, ambiguous field meaning

Output:

- **Match pairs** *(A row ↔ B row)* + optional confidence
- **Schema mapping** *(A.column → B.column)* + optional confidence

Data Discovery

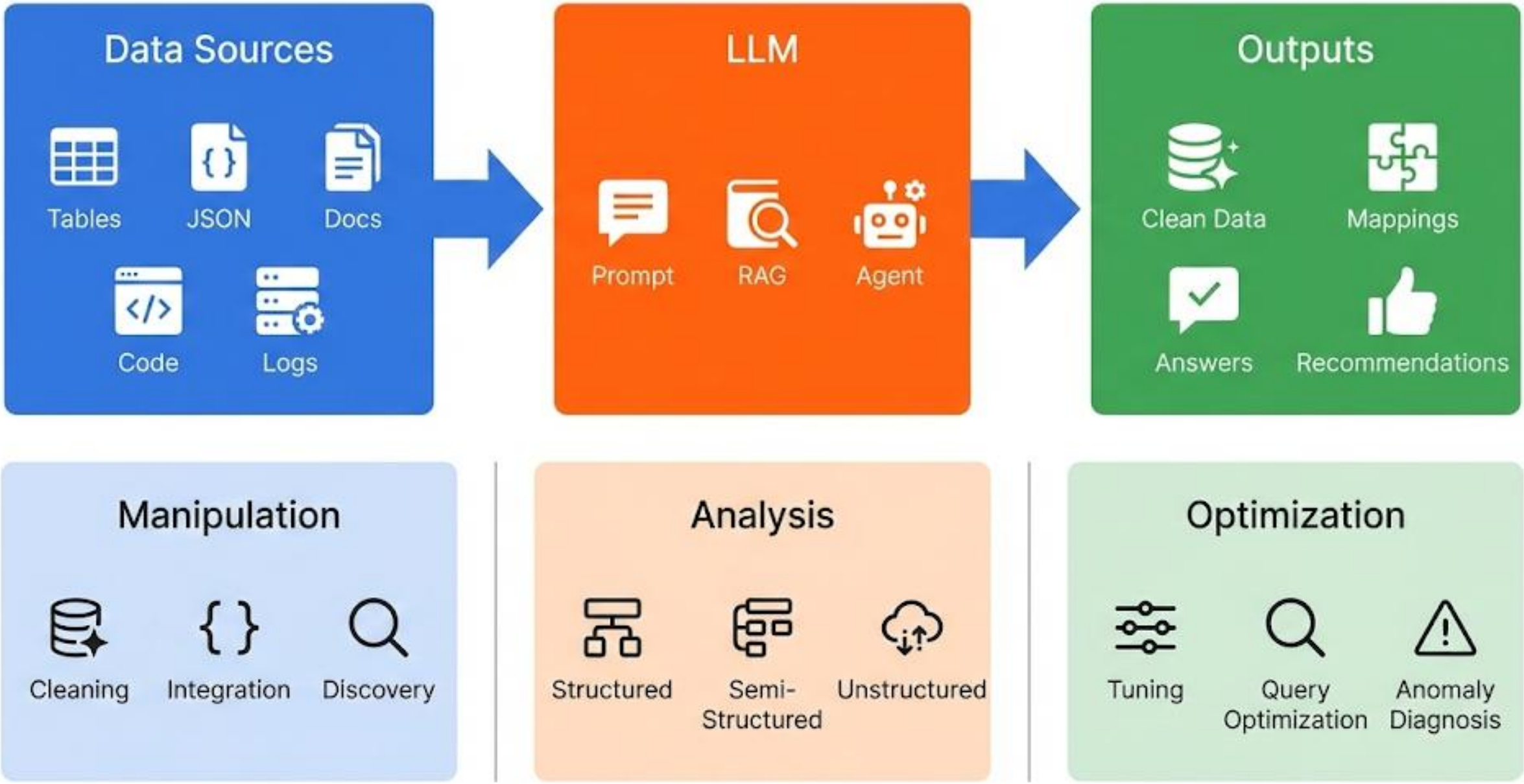
Goal: make datasets **understandable and searchable** before analysis

Two core sub-tasks:

- **Data Profiling:** summarize structure & quality
  (column types, value ranges, missingness, duplicates, examples)
- **Data Annotation:** assign semantic labels
  (entity type, column meaning, category tags, schema description)

Why it matters: without discovery, data exists but is **hard to reuse** (unknown semantics)

Input: dataset + (optional) metadata → **Output:** dataset card + labels/tags → improves reuse & downstream reliability


Data Sources
Tables
JSON
Docs
Code
Logs
LLM
Prompt
RAG
Agent
Outputs
Clean Data
Mappings
Answers
Recommendations
Manipulation
Cleaning
Integration
Discovery
Analysis
Structured
Semi-Structured
Unstructured
Optimization
Tuning
Query Optimization
Anomaly Diagnosis

LLM for Data Analysis

Goal: answer questions and perform reasoning over different data types

Three data categories:

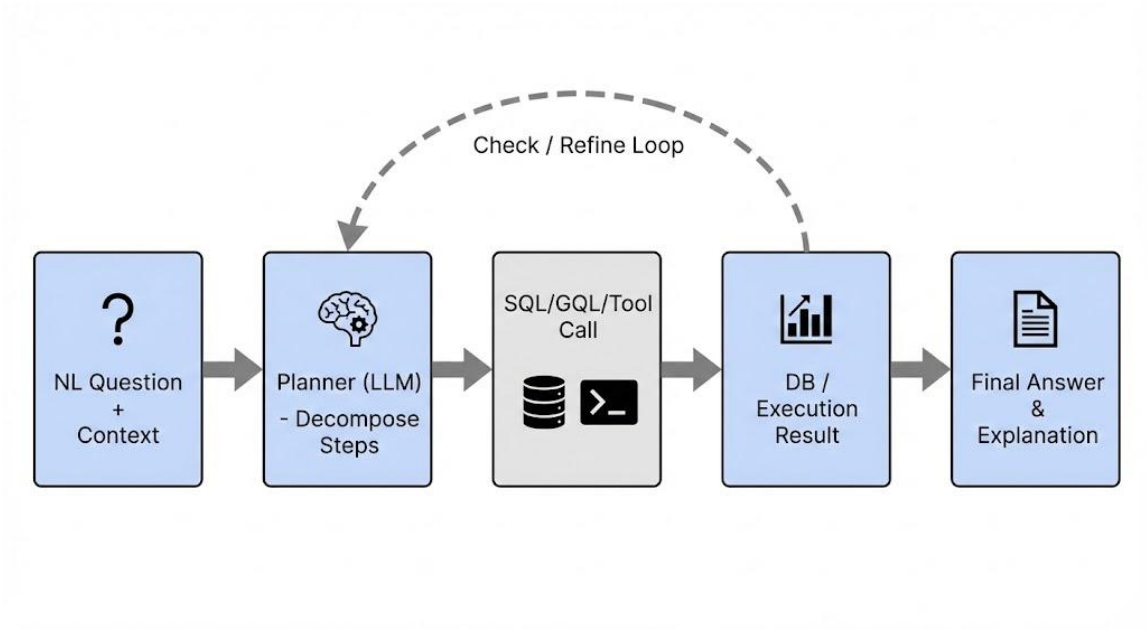
- **Structured:** relational tables & graphs (e.g., NL→SQL / QA)
- **Semi-structured:** JSON / HTML / XML / spreadsheets (often harder)
- **Unstructured:** documents (PDF/images) & programs (code repositories)

Key idea: as structure decreases, ambiguity increases → analysis becomes harder

Same template: Input (data type) → Reasoning (LLM + tools) → Output (answer / insight)

Structured Data Analysis

- **Input:** natural language question + table/graph
- **Approach:**
  - break the question into **steps**
  - generate **SQL / graph query** or call tools (Python, DB)
  - verify intermediate results
- **Output:** final answer + (optional) explanation / evidence



Semi-Structured Data

Examples: JSON, HTML, XML, spreadsheets (especially merged-cell tables)

Core issue: there is *some structure*, but it is **inconsistent or nested** → parsing + semantics are both needed

Common challenges:

- nested / hierarchical fields and schema variants
- merged cells and multi-level headers in spreadsheets
- structure can be **lost** when linearizing into plain text → ambiguity increases

Input: JSON/HTML/XML/Sheets → **LLM task:** parse + interpret → **Output:** correct fields + reliable answers/summaries

Unstructured Documents

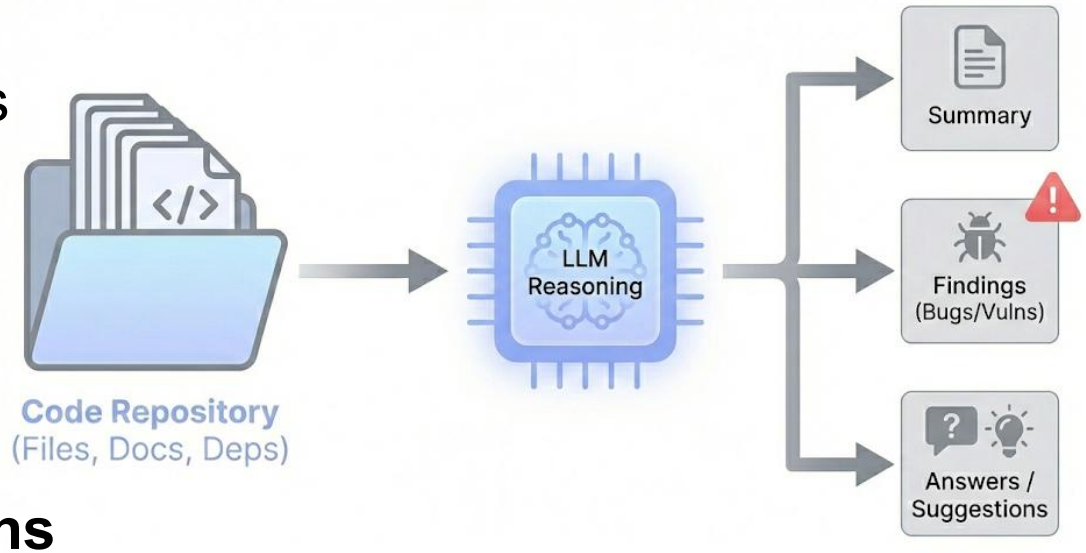
- **OCR-dependent pipeline:**
  Image/PDF → **OCR** → extracted text/layout → LLM reasoning → answer

- **OCR-free (end-to-end multimodal):**
  Image/PDF → **Multimodal LLM** → answer

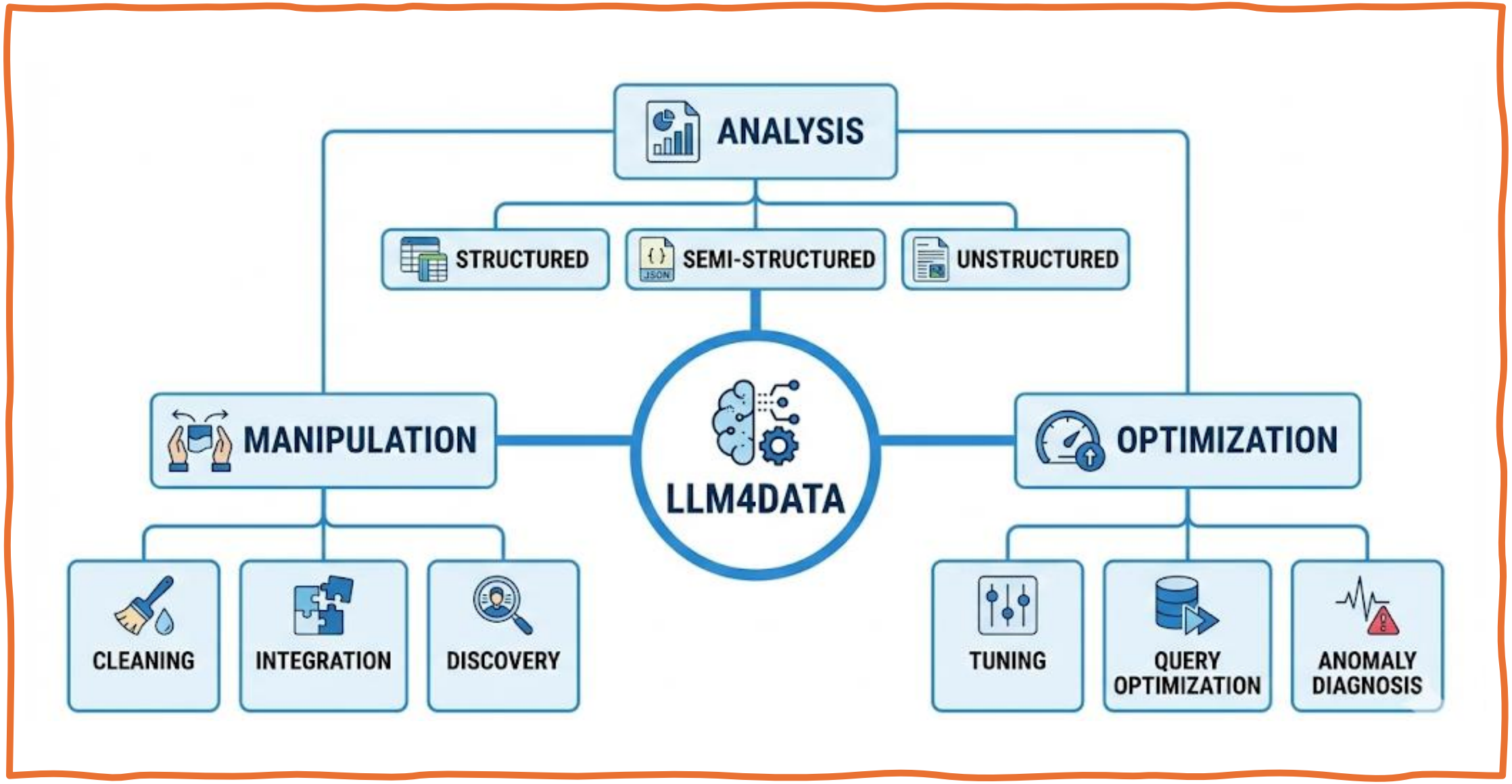
- **Trade-off:**
  OCR errors can propagate → reasoning errors; OCR-free reduces dependence on OCR

Unstructured Programs — Code Understanding & Analysis

- **Goal:** support software/data workflows using LLM reasoning over code
- Typical tasks:
  - **Repository summarization** (what does this project do?)
  - **Code QA / code search** (where is X implemented?)
  - **Bug/vulnerability detection** (identify risky patterns)
  - **Code completion / refactoring suggestions**



Challenge: reasoning at **repo-level context** (multiple files + dependencies)


ANALYSIS
STRUCTURED
SEMI-STRUCTURED
UNSTRUCTURED
MANIPULATION
LLM4DATA
OPTIMIZATION
CLEANING
INTEGRATION
DISCOVERY
TUNING
QUERY OPTIMIZATION
ANOMALY DIAGNOSIS

LLM for Data System Optimization

Goal: improve system **performance, cost, and reliability** using LLM-driven assistance

- Three core tasks:
  - **Configuration Tuning:** choose better system settings (knobs, indexes, resources)
  - **Query Optimization:** rewrite queries / improve plans for faster execution
  - **Anomaly Diagnosis:** detect and explain performance regressions or failures
- **Output:** actionable recommendations + (optional) explanations/evidence

Data System Optimization — What the LLM Does

- **Configuration Tuning**
  - **LLM role:** interpret workload signals, propose knob/index/resource changes, explain trade-offs
  - **Output:** recommended settings + expected impact
- **Query Optimization**
  - **LLM role:** rewrite queries, suggest plan-friendly structures, highlight inefficiencies
  - **Output:** optimized query (or rewrite suggestions) + brief rationale
- **Anomaly Diagnosis**
  - **LLM role:** summarize symptoms, correlate signals with changes, hypothesize root causes, recommend next checks
  - **Output:** ranked causes + next actions

Summary

- **LLM4DATA** is organized into three areas:
  - **Data Manipulation:** cleaning, integration, discovery
  - **Data Analysis:** structured, semi-structured, unstructured
  - **Data System Optimization:** tuning, query optimization, anomaly diagnosis
- Across all areas, LLMs act as a **semantic assistant**:
  - transform/align data,
  - reason over different data types,
  - convert system signals into actions

Outcome: more usable data + more reliable analysis + better system efficiency